SYSTEM DESIGN · INTERVIEW NOTES

System Design *Complete* Interview Notes

From scalability and caching to CAP, consistent hashing and estimation — the concepts an SDE interview probes, with tables, estimation formulas, solved numericals and rapid-fire revision.

36	80+	12	20+
SECTIONS	CONCEPTS	DESIGN WALKTHROUGHS	DESIGN PATTERNS

CONTENTS

01 Introduction to System Design
02 Scalability
03 Load Balancing
04 Caching
05 Databases: SQL vs NoSQL
06 Database Internals
07 Replication & Sharding
08 CAP Theorem & Consistency
09 Consistent Hashing
10 Message Queues & Event-Driven
11 API Design & Rate Limiting
12 Microservices vs Monolith
13 Distributed Systems Concepts
14 Storage & Search
15 Reliability & Availability
16 Observability
17 Back-of-the-Envelope Estimation
18 Common System Designs
19 The Interview Framework
20 Practice Numericals (Solved)
21 SOLID Principles
22 Design Patterns
23 LLD Fundamentals
24 LLD Walkthrough: Parking Lot
25 LLD Walkthrough: Elevator System
26 Design: URL Shortener
27 Design: Chat / Messaging
28 Design: Social / News Feed
29 Design: Rate Limiter
30 Design: Notification System
31 Design: Ride-hailing (Uber)
32 Design: Video Streaming (YouTube)
33 Design: Distributed Key-Value Store
34 Design: Search Autocomplete
35 Design: Web Crawler
36 Quick Revision Cheat-Sheet
37 Follow-up Drills

01 Introduction to System Design

DEFINITION

System design is the process of defining the architecture, components, modules, interfaces and data flow of a system so that it satisfies a set of functional and non-functional requirements.

System design is the process of defining the architecture, components, modules and data flow of a system to satisfy a set of requirements — while staying **scalable, reliable and maintainable**.

	HLD (HIGH-LEVEL DESIGN)	LLD (LOW-LEVEL DESIGN)
Focus	Architecture, components, data flow	Classes, functions, DB schema, APIs
Answers	"What are the parts and how do they talk?"	"How is each part built?"
Typical round	System design interview	Machine-coding / LLD round

Requirements — always start here

Functional — What the system *does* — e.g. "users can post a tweet and follow others".

Non-functional — How *well* it does it — scalability, availability, latency, consistency, durability.

GOLDEN RULE

In an interview, **ask questions first**. Clarify scope, scale and constraints before drawing anything. Never jump straight to a solution.

02 Scalability

DEFINITION

Scalability is a system's ability to handle growing load by adding resources — **vertically** (a bigger machine) or **horizontally** (more machines).

	VERTICAL SCALING (SCALE UP)	HORIZONTAL SCALING (SCALE OUT)
What	Add more power (CPU/RAM) to one machine	Add more machines
Limit	Hardware ceiling; single point of failure	Practically unlimited; fault-tolerant
Complexity	Simple, no code change	Needs load balancing, stateless services
Cost	Expensive at high end	Cheaper commodity machines

KEY IDEA

Horizontal scaling works cleanly only for **stateless** services — any server can handle any request. State is pushed to a shared store (DB, cache, object storage).

03 Load Balancing

DEFINITION

A **load balancer** is a component that distributes incoming traffic across multiple servers to improve availability, responsiveness and fault tolerance.

A **load balancer** distributes incoming traffic across multiple servers so no single server is overwhelmed.

ALGORITHM	HOW IT WORKS
Round Robin	Requests go to servers in turn.
Weighted Round Robin	Stronger servers get more requests.
Least Connections	Send to the server with fewest active connections.
IP Hash	Same client → same server (sticky sessions).
Least Response Time	Fastest server wins.

L4 vs L7 load balancing

- **L4** — routes by IP + port (TCP/UDP). Fast, no content awareness.
- **L7** — routes by URL, headers, cookies. Smarter (content-based), slightly slower.

Health checks let the LB stop sending traffic to a dead server. A good LB is itself made redundant so it is not a single point of failure.

04 Caching

> DEFINITION
>
> A **cache** is a high-speed storage layer that keeps frequently accessed data close at hand so future requests are served faster and the backend is spared.

A **cache** stores frequently used data in fast storage (RAM) to avoid slow work (DB query, network call). It is the single biggest lever for **latency** and **load**.

Where caches live

Client browser → CDN → API gateway → application (in-memory) → distributed cache (Redis/Memcached) → database.

Cache strategies

STRATEGY	HOW IT WORKS
Cache-aside (lazy)	App checks cache; on miss, reads DB and fills cache. Most common.
Write-through	Write to cache and DB together. Consistent, slower writes.
Write-back	Write to cache first, flush to DB later. Fast, risk of data loss.
Read-through	Cache itself fetches from DB on a miss.

Eviction policies: **LRU** (least recently used), **LFU** (least frequently used), **FIFO**, and **TTL** (expire after time).

HARD PARTS

Cache invalidation (keeping cache and DB in sync) and the **thundering herd** (many requests miss at once and hammer the DB). Fix with locking, staggered TTLs, or a single-flight pattern.

05 Databases: SQL vs NoSQL

DEFINITION

A **database** is an organised collection of data. **SQL** databases are relational (fixed schema, ACID); **NoSQL** databases are non-relational and built to scale horizontally.

	SQL (RELATIONAL)	NOSQL
Schema	Fixed, structured	Flexible / schemaless
Scaling	Mostly vertical; sharding is manual	Built to scale horizontally
Consistency	Strong (ACID)	Often eventual (BASE)
Joins	Powerful	Usually avoided (denormalised)
Use for	Banking, orders — where correctness matters	Big scale, flexible, high-write — feeds, logs, catalogs

Types of NoSQL

- **Key-Value** — Redis, DynamoDB (fast lookups).
- **Document** — MongoDB (JSON-like documents).
- **Column** — Cassandra, HBase (wide, write-heavy).
- **Graph** — Neo4j (relationships, social graphs).

06 Database Internals

DEFINITION

ACID (Atomicity, Consistency, Isolation, Durability) is the set of guarantees a relational transaction provides; an **index** is a data structure that speeds up reads.

ACID — relational guarantees

Atomicity	A transaction is all-or-nothing.
Consistency	The DB moves from one valid state to another.
Isolation	Concurrent transactions don't interfere.
Durability	Committed data survives crashes.

BASE (the NoSQL counterpart): Basically Available, Soft state, Eventual consistency.

Indexing

An **index** is a data structure (usually a **B-tree**) that speeds up reads at the cost of extra storage and slower writes. A **hash index** is ideal for exact-match lookups.

Normalization vs Denormalization

- **Normalization** — split data to remove redundancy (less storage, more joins).
- **Denormalization** — duplicate data for faster reads (common at scale).

07 Replication & Sharding

DEFINITION

Replication keeps copies of the same data on multiple nodes (for availability and read scaling); **sharding** splits data across nodes (for write scaling).

Replication — copies of the same data

- **Primary–Replica** (master–slave): writes go to the primary, reads spread across replicas. Most common.
- **Multi-primary**: multiple nodes accept writes (needs conflict resolution).

- **Synchronous** replication = consistent but slower; **asynchronous** = faster but may lag (replication lag).

Benefits: high availability, read scaling, and fault tolerance.

Sharding (partitioning) — splitting data across nodes

STRATEGY	HOW	RISK
Range	By key range (A–M, N–Z)	Hot spots
Hash	hash(key) mod N	Reshuffling when N changes
Directory	A lookup service maps key → shard	Extra hop, lookup SPOF

> **WHY IT MATTERS**
>
> Sharding enables scale beyond one machine, but choosing a good **shard key** is critical — a bad key creates uneven load (hot shards).

08 CAP Theorem & Consistency

> **DEFINITION**
>
> The **CAP theorem** states that a distributed system can guarantee only two of three properties simultaneously: **C**onsistency, **A**vailability and **P**artition tolerance.

CAP theorem: a distributed system can guarantee only **two** of these three during a network partition:

- **C**onsistency — every read sees the latest write.
- **A**vailability — every request gets a response.
- **P**artition tolerance — the system works despite dropped messages.

Since partitions *will* happen, real systems choose **CP** (e.g. HBase, MongoDB) or **AP** (e.g. Cassandra, DynamoDB). **PACELC** extends this: even with no partition, you trade **L**atency vs **C**onsistency.

Consistency models

MODEL	GUARANTEE
Strong	Reads always see the latest write.
Eventual	Replicas converge over time (fast, weaker).
Causal	Ordered only where cause–effect exists.

09 Consistent Hashing

DEFINITION

Consistent hashing maps keys and servers onto a ring so that adding or removing one server remaps only a small fraction (~1/N) of keys.

Plain `hash(key) mod N` breaks when you add/remove a server — almost every key remaps. **Consistent hashing** fixes this by placing both servers and keys on a **ring**; each key goes to the first server clockwise.

RULE

```
Adding/removing 1 of N servers remaps only about 1/N of the keys — not all of them.
```

EXAMPLE

With 4 servers, modulo hashing moves ~80% of keys when a 5th is added. Consistent hashing moves only ~1/5 = **20%**.

VIRTUAL NODES

To balance load, each physical server is mapped to many points (**virtual nodes**) on the ring. Used by Cassandra, DynamoDB, and CDNs.

10 Message Queues & Event-Driven

DEFINITION

A **message queue** is a buffer that lets producers send messages asynchronously while consumers process them later — decoupling the two and smoothing spikes.

A **message queue** decouples producers from consumers: the producer drops a message and moves on; the consumer processes it later. This enables **async processing**, smooths traffic spikes, and improves resilience.

Point-to-point	One message is consumed by exactly one consumer (work queue).
Pub–Sub	A message is delivered to all subscribers (topics).
Tools	Kafka (log/streaming), RabbitMQ (broker), SQS (managed queue).

Delivery guarantees

- **At-most-once** — may lose messages, never duplicates.
- **At-least-once** — never loses, may duplicate (so consumers must be **idempotent**).
- **Exactly-once** — ideal but expensive to guarantee.

Also: **dead-letter queues** for failed messages and **backpressure** when consumers fall behind.

11 API Design & Rate Limiting

DEFINITION

An **API** is a contract defining how software components communicate; **rate limiting** restricts how many requests a client may make in a given time window.

	REST	GRAPHQL	GRPC
Style	Resources + HTTP verbs	Query language, one endpoint	Remote procedure calls
Payload	JSON	JSON	Protobuf (binary)
Best for	Public APIs	Flexible clients	Fast internal microservices

Rate limiting — protecting against abuse

- **Token bucket** — tokens refill at a fixed rate; each request takes one. Allows bursts.
- **Leaky bucket** — requests drain at a constant rate (smooths bursts).
- **Fixed / sliding window** — allow N requests per time window.

Idempotency — an operation that can be repeated safely (e.g. `PUT`) without changing the result. Critical for retries.

12 Microservices vs Monolith

> DEFINITION
>
> A **monolith** is a single deployable application; **microservices** split it into small, independently deployable and scalable services.

	MONOLITH	MICROSERVICES
Structure	One deployable unit	Many small, independent services
Scaling	Scale the whole app	Scale only the hot service
Deploy	Simple but risky	Independent, frequent
Complexity	Low ops, hard to grow	High ops (network, tracing, data)

Supporting pieces: an **API gateway** (single entry point), **service discovery** (find instances), and **distributed tracing**. Start with a monolith and split only when the pain is real.

13 Distributed Systems Concepts

DEFINITION

A **distributed system** is a collection of independent computers that appears to its users as a single coherent system. **Quorum** and **consensus** help the nodes agree.

Quorum	For N replicas, a write quorum W and read quorum R give strong consistency when **W + R > N**.
Consensus	Getting nodes to agree on a value — algorithms: **Raft**, **Paxos**.
Leader election	Choosing one node to coordinate (used by consensus systems).
Two-Phase Commit	An atomic commit across nodes: prepare, then commit. Blocking.
Saga	A sequence of local transactions with compensating actions (the scalable alternative to 2PC).

EXAMPLE

With N = 3 replicas, choose W = 2 and R = 2 → W + R = 4 > 3, so reads always see the latest write.

14 Storage & Search

DEFINITION

Object storage stores data as blobs addressed by a key (e.g. S3); a **search engine** uses an **inverted index** to find documents by content.

TYPE	WHAT IT IS	EXAMPLE
Object storage	Store blobs (files, images) with a key	Amazon S3
Block storage	Raw volumes for VMs	EBS
File storage	Shared file system	EFS, NFS

Search & analytics

- **Elasticsearch** — uses an **inverted index** (word → documents) for fast full-text search.
- **Data warehouse** — structured analytics (Snowflake, Redshift).

- **Data lake** — raw data of any shape (S3 + Spark).

15 Reliability & Availability

> DEFINITION
>
> **Availability** is the proportion of time a system is operational (measured in "nines"); **reliability** is the probability it works correctly over a period.

Availability is expressed in "nines". Higher nines = less downtime but much more cost.

AVAILABILITY	DOWNTIME / YEAR	DOWNTIME / DAY
99%	3.65 days	14.4 min
99.9%	8.77 hours	1.44 min
99.99%	52.6 minutes	8.6 s
99.999%	5.26 minutes	0.86 s

How to get there

- **Redundancy** — no single point of failure.
- **Failover** — active-passive or active-active standby.
- **Graceful degradation** — shed non-critical features under load.

SLA (contract), **SLO** (target), **SLI** (measured indicator) define reliability goals.

16 Observability

> DEFINITION
>
> **Observability** is the ability to understand a system's internal state from its outputs — primarily **logs**, **metrics** and **traces**.

You cannot operate what you cannot see. The three pillars:

Logs	Timestamped records of events (debugging).
Metrics	Numeric time series — CPU, QPS, p99 latency (dashboards, alerts).
Traces	End-to-end path of one request across services (find the slow hop).

Tools: Prometheus + Grafana (metrics), ELK/OpenSearch (logs), Jaeger/Zipkin (traces). Good systems alert on **symptoms** (error rate, latency) not just causes.

17 Back-of-the-Envelope Estimation

DEFINITION

Back-of-the-envelope estimation is the practice of approximating scale (QPS, storage, bandwidth) with rough numbers to guide design decisions quickly.

Core formulas

ESTIMATIONS

```
QPS = requests per day ÷ 86,400  (use 86,400 s/day; ~100k/day ≈ 1 QPS)
Peak QPS = average QPS × peak factor (usually 2–3×)
Storage = record size × records × replication factor × time
Bandwidth = QPS × response size
Servers = peak QPS ÷ capacity per server (+ headroom)
Little's Law: L = λ × W (concurrent = arrival rate × time in system)
```

Latency numbers every engineer should know

OPERATION	APPROX. TIME
L1 cache reference	**1 ns**
Main memory (RAM) access	**100 ns**
SSD random read	**100 µs**
Disk seek	**~10 ms**
Round trip within a data centre	**~0.5 ms**
Round trip across continents	**~150 ms**

WHY

Interviews expect quick, roughly-correct maths. Round aggressively (86,400 → ~10^5, 1 day ≈ 10^5 s) and keep units consistent.

18 Common System Designs

URL Shortener (e.g. bit.ly)

Key generation (base62 of an auto-increment ID or a hash) → key-value store (short → long) → cache hot links → redirect (301/302). Reads far outweigh writes, so caching + read replicas matter.

Chat app (e.g. WhatsApp)

Persistent connections (WebSockets) → a message queue for delivery → per-user inbox store → presence service. Needs low latency and online/offline handling.

News feed (e.g. Twitter)

Fan-out on write (push to followers' feeds at post time — fast reads) vs **fan-out on read** (build the feed at request time — fast writes). Big accounts use a hybrid.

Rate limiter

Token bucket in a shared store (Redis) keyed by user/IP; return `429` when the limit is crossed.

Ride-hailing (e.g. Uber)

Location ingestion (high write) → geospatial index (geohash/quadtree) → matching service → trip state machine. Consistency matters for payments.

19 The Interview Framework

DEFINITION

The **system design interview** tests your ability to structure an ambiguous problem, make reasonable assumptions and reason about trade-offs — not to produce a perfect answer.

1. **Requirements** — functional + non-functional; ask clarifying questions.
2. **Estimation** — QPS, storage, bandwidth; state your assumptions aloud.
3. **High-level design** — draw clients, LB, services, cache, DB, queue.
4. **Deep dive** — pick 1–2 components the interviewer cares about and go deep.
5. **Bottlenecks & scale** — where will it break? How do you fix it?
6. **Wrap up** — trade-offs, what you'd monitor, what you'd do with more time.

> WHAT THEY'RE REALLY TESTING
>
> Not a perfect answer — but your **structured thinking**, ability to make **reasonable trade-offs**, and how you handle scale and failure.

20 Practice Numericals (Solved)

Q1 **A service has 10 million DAU, each making 20 requests/day. Find the average and peak QPS (peak = 3× average).**

SOLUTION

- Requests/day = 10M × 20 = 200M
- Avg QPS = 200,000,000 ÷ 86,400 ≈ **2,315**
- Peak = 3 × 2,315 ≈ **~7,000 QPS**

Q2 **Each request/response is 5 KB. Find the bandwidth at peak (7,000 QPS).**

SOLUTION

- Bandwidth = 7,000 × 5 KB = 35,000 KB/s
- = 35 MB/s = 35 × 8 = **280 Mbps**

Q3 **A service writes 100M records/day of 500 bytes each, replicated 3×. Storage per year?**

SOLUTION

- Per day = 100M × 500 B = 50 GB
- With replication = 50 × 3 = 150 GB/day
- Per year = 150 × 365 = **≈ 54.75 TB/year**

Q4 **What is the yearly downtime for 99.99% availability?**

SOLUTION

- Minutes/year = 365 × 24 × 60 = 525,600
- Downtime = 0.01% of that = 525,600 × 0.0001
- = **52.56 minutes/year**

Q5 **One server handles 1,000 QPS. How many servers for 50,000 QPS with 2× headroom?**

SOLUTION

- Servers = 50,000 ÷ 1,000 = 50
- With 2× headroom = 50 × 2 = **100 servers**

Q6 **Little's Law: arrival rate 2,000 req/s, average time in system 0.2 s. How many requests are in flight?**

SOLUTION

- $L = \lambda \times W = 2{,}000 \times 0.2$
- = **400 concurrent requests**

Q7 **By the 80/20 rule, 20% of data gets 80% of traffic. If the DB holds 1 TB, how big should the cache be to serve the hot set?**

SOLUTION

- Hot set = 20% of 1 TB
- = **200 GB**

Q8 **1 billion users spread over 1,000 shards. Users per shard?**

SOLUTION

- 1,000,000,000 ÷ 1,000 = **1,000,000 users/shard**

Q9 **Consistent hashing: with 4 servers, roughly what fraction of keys must move when a 5th server is added?**

SOLUTION

- Modulo hashing: almost all keys remap (~80%).
- Consistent hashing moves only ~1/N = 1/5
- = **~20% of keys**

Q10 **A chat app sends 50M messages/day of 100 bytes, replicated 3×, kept for 1 year. Storage?**

SOLUTION

- Per day = 50M × 100 B = 5 GB
- ×3 = 15 GB/day
- ×365 = **≈ 5.5 TB/year**

Q11 **An API must respond in 200 ms. The DB query takes 50 ms and serialization 20 ms. What is the budget left for network + compute?**

SOLUTION

- Remaining = 200 – 50 – 20
- = **130 ms**

Q12 **1 million concurrent video viewers at 5 Mbps each. Total egress bandwidth?**

SOLUTION

- 1,000,000 × 5 Mbps = 5,000,000 Mbps
- = 5,000 Gbps = **5 Tbps (why CDNs are essential)**

21 SOLID Principles

DEFINITION

SOLID is a set of five design principles — Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion — for maintainable OOP.

SOLID is five principles for writing maintainable, extensible object-oriented code — the backbone of low-level design.

S — Single Responsibility	A class should have **one reason to change** (one job).
O — Open/Closed	Open for extension, closed for modification — add features without editing existing code.
L — Liskov Substitution	A subclass must be usable anywhere its parent is, without surprises.
I — Interface Segregation	Many small interfaces beat one fat interface; don't force unused methods.
D — Dependency Inversion	Depend on abstractions, not concrete classes.

EXAMPLE

An `OrderService` that both calculates price *and* sends emails breaks **SRP** — split it into `PricingService` and `EmailService`. Injecting a `PaymentGateway` *interface* instead of a concrete `StripeGateway` satisfies **DIP**.

22 Design Patterns

DEFINITION

A **design pattern** is a reusable, named solution to a commonly occurring software design problem (e.g. Singleton, Factory, Observer, Strategy).

Reusable solutions to common design problems. Know the name, the problem it solves, and one example each.

Creational — how objects are created

PATTERN	USE
Singleton	Exactly one instance (config, connection pool).
Factory / Abstract Factory	Create objects without exposing the creation logic.
Builder	Build complex objects step by step.
Prototype	Clone existing objects.

Structural — how objects are composed

PATTERN	USE
Adapter	Make incompatible interfaces work together.
Decorator	Add behaviour dynamically by wrapping.
Facade	One simple interface over a complex subsystem.
Proxy	A stand-in that controls access to an object.
Composite	Tree structures (part–whole).

Behavioural — how objects interact

PATTERN	USE
Observer	Notify many objects of a change (pub–sub).
Strategy	Swap algorithms at runtime.
Command	Encapsulate a request as an object.
State	Behaviour changes with state (traffic light).
Iterator	Traverse a collection uniformly.
Template Method	Skeleton algorithm; subclasses fill in steps.

23 LLD Fundamentals

DEFINITION

Low-Level Design (LLD) focuses on the internal structure of components — the classes, interfaces, relationships and algorithms that implement a feature.

The four pillars of OOP

Encapsulation	Hide internal state; expose only what is needed.
Abstraction	Expose *what*, hide *how*.
Inheritance	Reuse and specialise (is-a).
Polymorphism	Same call, different behaviour (overloading / overriding).

Relationships (UML)

Association (uses) · **Aggregation** (has-a, weak) · **Composition** (owns, strong) · **Inheritance** (is-a).

Concurrency basics

Shared mutable state causes **race conditions**. Guard it with locks/mutexes, or avoid it with immutability. Know thread-safe collections and **deadlock** (circular waiting on locks).

LLD INTERVIEW APPROACH

1) Clarify requirements → 2) identify **entities** (the nouns) → 3) define **interfaces & responsibilities** → 4) map **relationships** → 5) code the key classes → 6) discuss extensibility & concurrency.

24 LLD Walkthrough: Parking Lot

Requirements

Multiple levels, spot types (bike / car / truck), issue a ticket on entry, compute the fee on exit, support multiple payment methods.

Entities → classes

- `ParkingLot` — holds `Level[]`, methods `entry()`, `exit()`
- `Level` — holds `ParkingSpot[]`, `findSpot(type)`

- `ParkingSpot` — type, occupied, `assign(vehicle)`, `free()`
- `Vehicle` (Bike / Car / Truck), `Ticket` (entry time, spot), `Payment`

Key design decisions

- **Strategy pattern** for payment (UPI / card / cash) and for pricing (hourly / flat).
- Spot selection can use a per-type free list for O(1) allocation.
- Thread-safety: lock a spot while assigning it so two cars cannot take it.

> EXTENSIONS THEY MAY ASK
>
> Reserve a spot in advance, EV charging spots, dynamic pricing by demand, multiple entry gates — each maps to a new class or strategy, not a rewrite.

25 LLD Walkthrough: Elevator System

Requirements

N elevators, external up/down buttons per floor, internal floor buttons, efficient scheduling, direction and state tracking.

Entities → classes

- `ElevatorSystem` — manages elevators and dispatches requests
- `Elevator` — current floor, direction, state (IDLE / MOVING / DOOR_OPEN), pending requests
- `Request` — floor + direction; `Direction` enum (UP / DOWN / IDLE)

Scheduling

Use the **SCAN / LOOK** algorithm — an elevator keeps moving in one direction, serving all requests on the way, then reverses. Pick the best elevator by distance and direction.

> TALKING POINT
>
> The trade-off between *wait time* and *travel time*, and modelling **state transitions** cleanly with the State pattern.

26 Design: URL Shortener

Requirements

Functional: shorten a long URL, redirect, custom alias, expiry, basic analytics. Non-functional: highly available, low latency, read-heavy (roughly **100:1** read:write).

Estimation

> QUICK MATHS
> 100M new URLs/day → ~**1,160 writes/s**. Reads ~100× → ~**116k reads/s**. Storage ≈ 100M × 500 B ≈ 50 GB/day.

Key generation

Base62 (`a–z A–Z 0–9`) of a global counter gives short, unique keys (7 chars → $62^7 \approx 3.5$ trillion combinations).

Components

Client → Load Balancer → App servers → **KV store** (short → long URL) with a **cache** for hot links → analytics (async via a queue).

Trade-offs

- **301** (permanent) is cacheable but loses analytics; **302** (temporary) keeps analytics. Most shorteners use 302.
- Counter-based keys need a distributed counter (or ranges per node) to avoid collisions.

27 Design: Chat / Messaging

Requirements

1-1 and group chat, delivery and read receipts, presence (online/offline), message history. Non-functional: low latency, high availability, ordered delivery per conversation.

Components

- **WebSocket gateways** — hold persistent connections to online users.
- **Message service** → writes to a **message store** (e.g. Cassandra, keyed by conversation).

- **Queue** for async fan-out; a **presence service** (heartbeats); **push** (FCM/APNS) for offline users.

Scaling & bottlenecks

- Connection servers scale horizontally; route a user to the same server via a session registry.
- Group fan-out is the hard part — for very large groups, fan-out on read.
- Ordering: use sequence numbers per conversation.

28 Design: Social / News Feed

Requirements

Post content, follow users, see a ranked feed. Non-functional: low-latency reads, huge scale, eventual consistency is acceptable.

The core trade-off

APPROACH	HOW	GOOD FOR
Fan-out on write (push)	On post, push it into every follower's feed cache.	Fast reads; bad for celebrities (huge fan-out).
Fan-out on read (pull)	Build the feed at request time from followees' posts.	Fast writes; slower reads.
Hybrid	Push for normal users, pull for celebrities.	Real systems (Twitter / Instagram).

Components

Post service → post store; **graph service** (follows); **feed service** + cache; a queue for fan-out; a ranking service.

29 Design: Rate Limiter

Requirements

Limit requests per user / IP / API key; return `429` when exceeded; distributed and low-latency.

Algorithms

ALGORITHM	NOTES
Token bucket	Tokens refill at a rate; allows short bursts. Most common.
Leaky bucket	Constant outflow; smooths traffic.
Fixed window	Simple, but allows a 2× burst at window boundaries.
Sliding window	Accurate; uses more memory.

Design

Place it at the **API gateway** or as middleware. Keep counters in a shared store (**Redis**) using atomic ops (`INCR` + `EXPIRE`). Return headers such as `X-RateLimit-Remaining` .

30 Design: Notification System

Requirements

Send notifications over multiple channels (push, SMS, email), respect user preferences, retry failures, avoid spam. Non-functional: reliable, scalable, asynchronous.

Components

API → **queue** → channel workers (push / SMS / email) → third-party providers (FCM, Twilio, SES). Plus: template service, preference service, rate limiting, **retry with backoff**, and a **dead-letter queue**.

Key points

- Decouple with a queue so a slow channel never blocks the others.
- Idempotency — don't send the same notification twice on retries.
- Priority queues for OTP vs marketing.

31 Design: Ride-hailing (Uber)

Requirements

Riders request a trip; match the nearest driver; live location tracking; trip state; payment. Non-functional: very high write (location pings), low-latency matching, consistency for payments.

Components

- **Location service** — ingests driver pings (high write); stores the latest location.
- **Geospatial index** — geohash / quadtree to find nearby drivers fast.
- **Matching service** — picks a driver (nearest / best score).
- **Trip service** — a state machine (REQUESTED → MATCHED → ONGOING → COMPLETED).
- **Payment** — consistent and idempotent; surge pricing.

> HARD PART
> Location writes are massive (millions of drivers pinging every few seconds) — use in-memory / geo stores and drop stale pings.

32 Design: Video Streaming (YouTube)

Requirements

Upload, store and stream videos at scale; adaptive quality; low startup latency. Reads (viewing) far exceed writes (upload).

Pipeline

Upload → object storage (raw) → **transcoder** creates multiple resolutions / formats (HLS, DASH) → stored in **object storage** → served via **CDN** → metadata and recommendations in a DB.

Key points

- Transcoding is heavy — do it asynchronously via a queue + worker fleet.
- Adaptive bitrate: the player switches quality based on bandwidth.

- CDN is essential — egress is enormous (see the 5 Tbps estimation).

33 Design: Distributed Key-Value Store

Goal

A DynamoDB-style store: partition and replicate data across nodes, stay available, and resolve conflicts.

Design decisions

- **Partitioning** — **consistent hashing** spreads keys across nodes.
- **Replication** — each key is stored on N nodes; tune the **quorum** (W + R > N) for consistency.
- **Conflict resolution** — vector clocks or last-write-wins.
- **Failure handling** — hinted handoff, anti-entropy (Merkle trees), gossip for membership.

EXAMPLE

N = 3, W = 2, R = 2 → W + R = 4 > 3 → strong consistency. Choose W = 1, R = 1 for speed (eventual consistency).

34 Design: Search Autocomplete

Requirements

Return top-K suggestions for a typed prefix, extremely fast, ranked by popularity.

Design

- **Trie** (prefix tree) where each node caches its **top-K** queries — giving O(prefix length) lookup.
- Popularity aggregated by a data pipeline from search logs; the trie is rebuilt / updated offline.
- **Cache** hot prefixes; shard the trie by prefix range for scale.

TRADE-OFF

Storing top-K at every node speeds up reads but uses more memory — a classic space vs speed trade-off.

35 Design: Web Crawler

Requirements

Download billions of pages, respect robots.txt & politeness, avoid duplicates, keep content fresh. Distributed and fault-tolerant.

Components

- **URL Frontier** — a priority queue (importance + freshness) with per-host queues for politeness.
- **DNS resolver** cache, **fetcher** (HTTP client pool).
- **Parser** — extracts links & content; **dedup** via a **Bloom filter** (seen URLs / content hashes).
- **Storage** — raw pages in object storage; an index for search.

HARD PARTS

Politeness (don't hammer one site), trap pages (infinite calendars), and freshness scheduling (re-crawl by change rate).

36 Quick Revision Cheat-Sheet

- Always start with **requirements** (functional + non-functional), then estimate, then design.
- Vertical = scale up (limited); horizontal = scale out (needs stateless services).
- LB: round-robin / least-connections; L4 (IP/port) vs L7 (content).
- Cache: cache-aside is default; evict with LRU/LFU/TTL; hard part = invalidation.
- SQL = ACID, joins, vertical; NoSQL = flexible, horizontal, BASE.
- Replication = copies (read scaling); Sharding = split (write scaling).

- CAP: pick CP or AP under a partition; PACELC adds latency vs consistency.
- Consistent hashing moves only ~1/N keys when a node changes.
- Queues decouple services; make consumers idempotent (at-least-once delivery).
- Rate limiting: token bucket, leaky bucket, sliding window.
- Availability: 99.9% = 8.77 h/yr, 99.99% = 52.6 min/yr.
- Estimation: QPS = req/day ÷ 86,400; Little's Law L = λW.

> **DON'T FORGET**
>
> Every design must answer: **How does it scale? Where does it fail? How is data kept consistent?** Trade-offs, not buzzwords, win the interview.

37 Follow-up Drills

Interviewers rarely stop at the first answer. Each drill below is the main question, then the follow-ups they actually push on — practise the whole chain, not just the first line.

Q How do you decide between vertical and horizontal scaling?

↳ **Which breaks first when you scale out?**

Usually shared state — the database, sessions, and file storage. That is why the first moves are stateless services plus externalised session/cache storage.

↳ **What is the practical limit of vertical scaling?**

Cost and single-point-of-failure: bigger machines get disproportionately expensive and you still have one machine to lose.

Q What caching strategy would you use, and why?

↳ How do you invalidate the cache?

TTL for tolerance of staleness, explicit invalidation on write, or write-through if consistency matters more than latency. The choice is a consistency-versus-latency trade.

↳ What is a cache stampede and how do you prevent it?

Many clients miss simultaneously and hammer the database. Fix with request coalescing (single-flight), jittered TTLs, or a short lock around the refill.

Q Explain CAP with a real system example.

↳ Give a CP and an AP example.

CP: a payments or inventory system that refuses writes during a partition rather than risk inconsistency. AP: a social feed or DNS that keeps serving possibly stale data and reconciles later.

↳ What actually happens during a partition?

You must choose: reject some requests to stay consistent (CP), or accept and reconcile later (AP). There is no third option while the network is split.

Q How do you choose a shard key?

↳ What is a hot partition and how do you avoid it?

One shard gets disproportionate traffic (a celebrity user, a timestamp). Fix by hashing the key, adding a salt, or splitting that key into sub-shards.

↳ Why is resharding painful?

It means moving data while serving traffic. Consistent hashing limits how much moves, but you still need a migration path and dual-read or dual-write during the move.

Q Why consistent hashing instead of modulo hashing?

↳ Why are virtual nodes used?

They smooth the distribution: each physical node owns many small ranges, so load is balanced and a node failure redistributes evenly.

↳ What problem does modulo hashing cause?

Changing the number of nodes remaps almost every key, so nearly the entire cache or dataset must move at once.

Q Design a rate limiter — which algorithm?

↳ **Token bucket vs leaky bucket?**

Token bucket allows short bursts up to the bucket size while enforcing an average rate. Leaky bucket smooths output to a constant rate. Bursty APIs usually want token bucket.

↳ **How do you make it distributed?**

Keep counters in a shared store like Redis with atomic increment and expiry, accepting a little network latency per check; local-only counters would multiply the limit by the number of instances.

Q How do you guarantee a message is processed exactly once?

↳ **At-least-once vs exactly-once — what is realistic?**

Exactly-once delivery is effectively at-least-once plus idempotent consumers. Most systems guarantee at-least-once and make handlers idempotent.

↳ **How do you handle a poison message?**

After N failed attempts, move it to a dead-letter queue so it stops blocking the partition, and alert on DLQ depth.

Q Design a URL shortener — how do you generate IDs?

↳ **How do you avoid collisions at scale?**

Use a distributed ID generator (Snowflake-style), a counter service with pre-allocated ranges, or hash with a collision check and retry.

↳ **How do you handle redirects at scale?**

The mapping is read-heavy and immutable, so cache aggressively at the edge; the database is only the source of truth.

Q Design a chat system — how do you deliver to offline users?

↳ **How do you order messages correctly?**

Per-conversation sequence numbers from a single writer (or a partitioned log) give a total order within a chat without a global lock.

↳ **How do you handle presence and fan-out?**

Presence is ephemeral state in Redis; fan-out uses long-lived sockets with a pub/sub bus so any instance can reach the right connection.

Q Design video streaming — why HLS over progressive download?

> ↳ **How do you store and transcode at scale?**
>
> Store the original once, transcode asynchronously into a bitrate ladder on worker fleets, and serve segments from object storage behind a CDN so no app server carries video bytes.

> ↳ **Where is the bottleneck in this design?**
>
> Transcoding CPU and CDN egress cost. That is why you transcode once per rendition, cache segments at the edge, and gate quality by plan.